Erlind Sejdiu
106357

SYCL

Portable heterogene Programmierung mit C++

Problemstellung, Anwendungsfälle und Programmiermodell

Seminararbeit
der Hochschule Karlsruhe
im Master Studiengang Informatik
im Wintersemester 2026/27

Betreuer: [Name der betreuenden Person]
Themengruppe: [Themengruppe des Seminars]
Abgabedatum: [Datum]

Inhaltsverzeichnis

1 Einleitung **1**

2 Problemstellung **1**

3 SYCL als Lösungsansatz **1**

4 Anwendungsfälle **2**

5 Das Programmiermodell **2**

5.1 Plattformen, Geräte und Geräteauswahl 2

5.2 Queues, Command Groups und Abhängigkeiten 3

5.3 Ausführungsmodell 4

5.4 Speichermodell 5

5.5 Fehlerbehandlung, Spezialisierung und Interoperabilität 6

6 Anwendungsbeispiel: Matrixmultiplikation **7**

7 Bewertung und Fazit **8**

Literaturverzeichnis **9**

Abbildungsverzeichnis

1 Mehrere herstellerspezifische Codebasen (links) gegenüber einer SYCL-Codebasis, die über Backends wie CUDA, HIP, Level Zero oder OpenCL auf verschiedener Hardware läuft (rechts) 1
2 Geräteauswahl: Der Selector bewertet jedes Gerät, die Queue wird an das bestbewertete gebunden 3
3 Abhängigkeitsgraph: Die Kopie von D kann parallel zu Kernel 1 laufen 4
4 Gliederung des Indexraums in Work-Groups und Work-Items 5
5 Typische Abbildung der SYCL-Ausführungshierarchie auf GPU und CPU 5
6 Buffer-Modell: Aus den Accessoren leitet die Laufzeit Kopien und Abhängigkeiten selbst ab (orange) 5
7 Speicherhierarchie von Host und Gerät und die Wege des Datentransfers 6
8 Synchrone Fehler werden beim Einreichen geworfen, asynchrone erst, wenn der Host wartet 7
9 Datentransfer (a) und Leistung je Kachelgröße mit Performance-Portabilität (b) 8

Tabellenverzeichnis

1 SYCL im Vergleich zu anderen Programmiermodellen ((✓) = eingeschränkt) 2
2 Anwendungsfälle, ihre Anforderungen und die Mechanismen, die sie erfüllen 3
3 Die beiden Speichermodelle von SYCL 6

1 Einleitung

Rechenleistung wächst heute vor allem durch Spezialisierung: Neben der CPU stehen in fast jedem Rechner GPUs und andere Beschleuniger, die datenparallele Aufgaben um Größenordnungen schneller bearbeiten. Die Software hält mit dieser Vielfalt jedoch nur schwer Schritt, denn jeder Hersteller bringt sein eigenes Programmiermodell mit. SYCL ist ein offener Standard der Khronos Group, der dieses Problem lösen will: Ein einziger Quellcode in Standard-C++ soll auf CPUs und GPUs verschiedener Hersteller laufen [1].

Diese Arbeit erklärt, welches Problem SYCL adressiert, für welche Anwendungsfälle es gedacht ist und wie sein Programmiermodell diese erfüllt. Ein kurzes Beispiel macht die Konzepte greifbar.

2 Problemstellung

Heutige Systeme kombinieren CPUs mit GPUs unterschiedlicher Hersteller. Für die Software ergeben sich daraus drei Probleme:

1. **Herstellerbindung:** CUDA läuft nur auf NVIDIA-GPUs [2]; AMD (HIP) und Intel (Level Zero) haben eigene Schnittstellen.
2. **Mehrfache Codebasen:** Wer mehrere Hersteller unterstützen will, pflegt mehrere Implementierungen parallel (Abbildung 1, links).
3. **Abstraktion gegen Kontrolle:** Offene Alternativen sind entweder sehr hardwarenah und aufwendig wie OpenCL [3] oder lassen wie OpenMP-Direktiven wenig Kontrolle über Speicher und Ausführung.

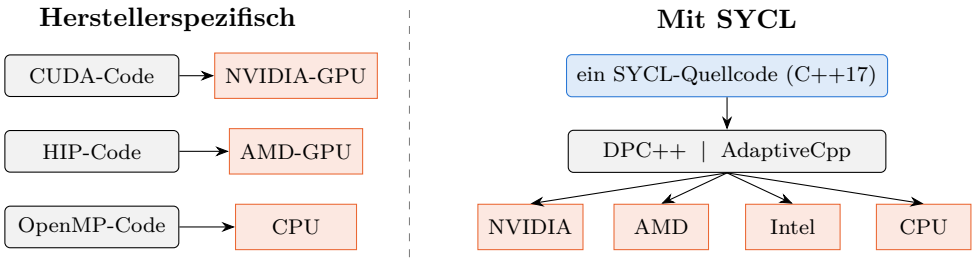


Abbildung 1: Mehrere herstellerspezifische Codebasen (links) gegenüber einer SYCL-Codebasis, die über Backends wie CUDA, HIP, Level Zero oder OpenCL auf verschiedener Hardware läuft (rechts)

3 SYCL als Lösungsansatz

SYCL ist ein offener Standard der Khronos Group auf Basis von C++17 [1]. Implementierungen wie DPC++ [4] und AdaptiveCpp [5], [6] bilden ihn über Backends wie

CUDA, HIP, Level Zero, OpenCL oder OpenMP auf die Hardware ab und übersetzen den Gerätecode meist erst beim ersten Start *just in time* für das konkrete Gerät [7]. Den drei Problemen setzt SYCL vier Entwurfsziele entgegen:

Portabilität: Ein Quellcode, viele Backends; das Gerät wird erst beim Übersetzen bzw. zur Laufzeit gewählt.

Offenheit: Ein offener Standard mit mehreren unabhängigen Implementierungen statt eines Herstellerprodukts.

Produktivität: *Single-Source* in Standard-C++: Host- und Gerätecode stehen in einer Datei, Kernel sind gewöhnliche Lambdas, und Templates und Typen werden geteilt.

Kontrolle: Wo nötig, legt das Modell Hardwarekonzepte wie Work-Groups und lokalen Speicher offen, um gezielt optimieren zu können.

Tabelle 1 vergleicht SYCL entlang dieser Ziele mit den Alternativen. Kokkos [8] und RAJA [9] verfolgen ähnliche Ziele, sind aber kein offener Industriestandard.

	herstellerunabh.	**offener Standard**	**Standard-C++**	**Kontrolle**
CUDA	–	–	(✓)	✓
OpenCL	✓	✓	–	✓
OpenMP-Offload	✓	✓	✓	(✓)
Kokkos/RAJA	✓	–	✓	(✓)
SYCL	✓	✓	✓	✓

Tabelle 1: SYCL im Vergleich zu anderen Programmiermodellen ((✓) = eingeschränkt)

4 Anwendungsfälle

Aus den Entwurfszielen ergeben sich vier typische Einsatzfelder (Tabelle 2): wissenschaftliches Rechnen auf Systemen verschiedener Hersteller, etwa auf dem Supercomputer Aurora mit Intel-GPUs oder in der Molekulardynamik-Software GROMACS [10], [11]; die Migration bestehender CUDA-Anwendungen, unterstützt durch das Werkzeug SYCLomatic; die gemeinsame Nutzung von CPU und GPU in einem Rechner; und portable Bibliotheken wie oneMKL oder oneDPL [12]. Die Tabelle zeigt, welche Anforderung jedes Feld stellt und welcher Teil des Programmiermodells sie erfüllt.

5 Das Programmiermodell

5.1 Plattformen, Geräte und Geräteauswahl

SYCL unterscheidet den *Host*, auf dem das Hauptprogramm läuft, und *Geräte*, auf denen Kernel ausgeführt werden. Geräte sind nach *Plattformen* gruppiert, wobei jede

Anwendungsfall	Anforderung	Antwort im Programmiermodell
Rechnen über Herstellergrenzen	ein Code für viele Systeme, hohe Leistung	Backends und Geräteauswahl (5.1); hardwarenahe Hierarchie (5.3); Interoperabilität mit Herstellerbibliotheken (5.5)
Migration von CUDA	vertraute Konzepte, schrittweiser Umbau	Work-Group/Sub-Group analog zu Block/Warp (5.3); zeigerbasiertes USM (5.4)
CPU und GPU gemeinsam	Überlappung, Ausweichgerät	asynchrone Queues und Abhängigkeitsgraph (5.2); CPU als Gerät (5.1)
Portable Bibliotheken	generischer Code, Anpassung je Gerät	Single-Source mit Templates (3); `get_info`, Aspekte, Spezialisierungskonstanten (5.1, 5.5)

Tabelle 2: Anwendungsfälle, ihre Anforderungen und die Mechanismen, die sie erfüllen

Plattform einem Backend entspricht; auch die CPU kann als Gerät auftreten. Welches Gerät verwendet wird, bestimmt ein *Device Selector*. Vordefiniert sind Selektoren für das als am leistungsfähigsten eingeschätzte Gerät, für GPUs und für CPUs; eigene Selektoren vergeben Punktwerte anhand von Geräteeigenschaften. Über *Aspekte* kann ein Programm zudem prüfen, ob ein Gerät optionale Fähigkeiten wie doppelte Genauigkeit oder bestimmte USM-Arten unterstützt, und mit `get_info` Eigenschaften wie die maximale Work-Group-Größe abfragen [1]. Implementierungen erlauben außerdem eine Auswahl von außen, etwa über Umgebungsvariablen, sodass derselbe Code ohne Neuübersetzung auf einem anderen Gerät läuft. Dass auch die CPU als vollwertiges Gerät gilt, macht sie zum Ausweichgerät, wenn kein Beschleuniger vorhanden ist, und erlaubt die Entwicklung ohne GPU. Abbildung 2 veranschaulicht die Auswahl.

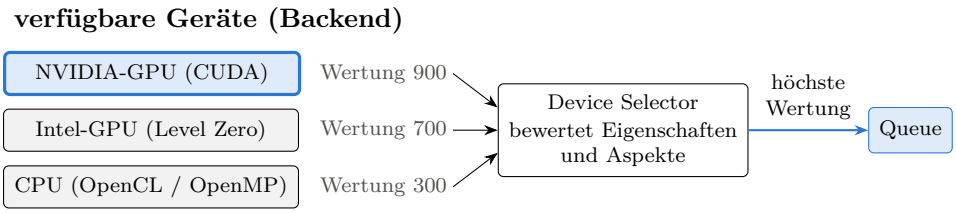


Abbildung 2: Geräteauswahl: Der Selector bewertet jedes Gerät, die Queue wird an das bestbewertete gebunden

5.2 Queues, Command Groups und Abhängigkeiten

Die Verbindung zu einem Gerät stellt eine *Queue* her. Arbeit wird in Form von *Command Groups* eingereicht: Eine Command Group beschreibt eine Aktion, etwa einen Kernel oder eine Kopie, zusammen mit den Daten, die sie benötigt. Technisch ist eine

Command Group eine Funktion, die ein *Handler*-Objekt erhält: Über den Handler deklariert sie ihre Datenzugriffe und Abhängigkeiten und startet genau eine Aktion. Das Einreichen kehrt sofort zurück und liefert ein *Event*. Die Ausführung ist also asynchron, und der Host wartet nur, wenn er es ausdrücklich verlangt. Abhängigkeiten zu früheren Aufträgen lassen sich über deren Events angeben. Mit *Host-Tasks* kann zudem gewöhnlicher Host-Code als Knoten in den Graphen eingereiht werden, etwa um nach einer Berechnung eine Datei zu schreiben, ohne den Rest des Programms zu blockieren.

Aus den Datenzugriffen und Events baut die Laufzeitumgebung einen gerichteten azyklischen Abhängigkeitsgraphen auf (Abbildung 3). Voneinander unabhängige Aufträge darf sie parallel oder in anderer Reihenfolge ausführen, abhängige startet sie erst, wenn ihre Vorgänger fertig sind [12]. So lassen sich Datentransfers, Berechnungen und Host-Arbeit überlappen, ohne Threads oder Streams von Hand zu verwalten. Wer eine strikte Reihenfolge bevorzugt, legt die Queue als *in-order* an.

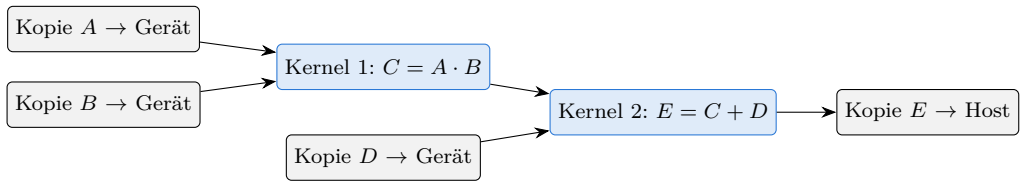


Abbildung 3: Abhängigkeitsgraph: Die Kopie von D kann parallel zu Kernel 1 laufen

5.3 Ausführungsmodell

Ein datenparalleler Kernel wird mit `parallel_for` für jeden Punkt eines ein- bis dreidimensionalen Indexraums einmal ausgeführt; eine einzelne Ausführung heißt *Work-Item*. Bei einer einfachen `range` gibt der Programmierer nur die Gesamtgröße an und überlässt die Aufteilung der Implementierung. Bei einer `nd_range` legt er zusätzlich fest, wie die Work-Items zu *Work-Groups* gebündelt werden (Abbildung 4). Die Work-Items einer Gruppe laufen gemeinsam auf einer Recheneinheit, teilen sich schnellen lokalen Speicher und können sich über Barrieren synchronisieren. Seit SYCL 2020 gibt es darunter *Sub-Groups*, deren Work-Items im Gleichschritt laufen und Daten direkt austauschen können [1].

Diese Hierarchie bildet die Hardware ab (Abbildung 5) und entspricht weitgehend dem Modell von CUDA [2]: Auf einer NVIDIA-GPU wird eine Work-Group zu einem Thread-Block mit höchstens 1024 Threads [13], eine Sub-Group zu einem Warp. Auf einer CPU arbeitet typischerweise ein Kern eine Work-Group ab, und die Sub-Group entspricht den Lanes der Vektoreinheit. Neben `parallel_for` bietet SYCL 2020 fertige Gruppenalgorithmen, etwa um innerhalb einer Work-Group einen Wert zu verteilen oder zu summieren, sowie *Reduktionen*, mit denen sich Ergebnisse aller Work-Items,

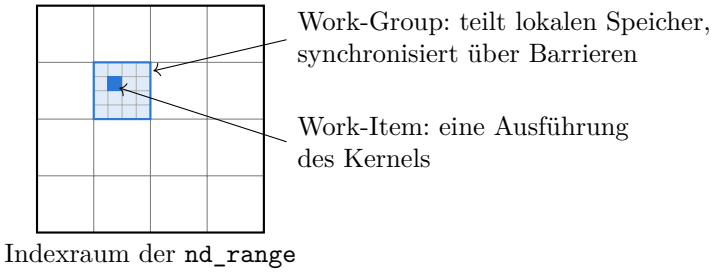


Abbildung 4: Gliederung des Indexraums in Work-Groups und Work-Items

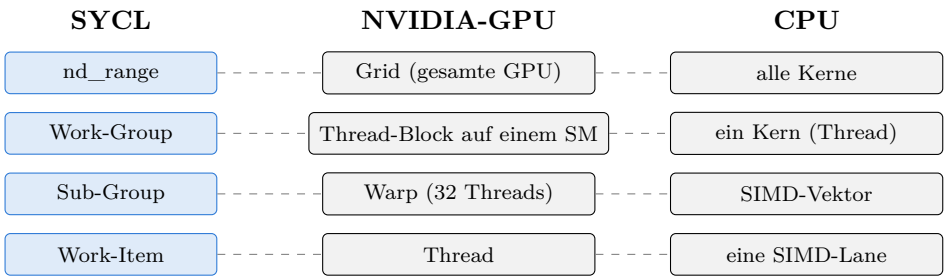


Abbildung 5: Typische Abbildung der SYCL-Ausführungshierarchie auf GPU und CPU

etwa eine Summe oder ein Maximum, ohne eigene Synchronisation zusammenfassen lassen.

Für den Gerätecode gelten Einschränkungen, weil er auf Hardware ohne vollständige C++-Laufzeitumgebung laufen muss: Ausnahmen, virtuelle Funktionsaufrufe, Rekursion und dynamische Speicherreservierung sind im Kernel nicht erlaubt, und Daten werden per Wertkopie übernommen [1].

5.4 Speichermodell

GPUs besitzen meist eigenen Speicher, der vom Hauptspeicher des Hosts getrennt ist. SYCL bietet zwei Modelle, um Daten zwischen beiden zu bewegen (Tabelle 3).

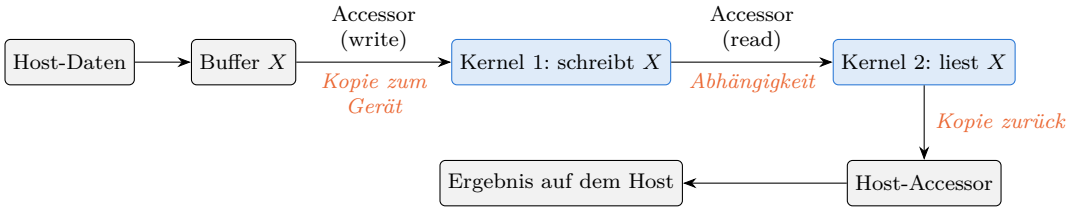


Abbildung 6: Buffer-Modell: Aus den Accessoren leitet die Laufzeit Kopien und Abhängigkeiten selbst ab (orange)

Beim *Buffer-Modell* kapselt ein Buffer die Daten, und Kernel greifen über *Accessoren* darauf zu, die angeben, ob gelesen oder geschrieben wird. Daraus leitet die Laufzeit

	Buffer und Accessoren	**Unified Shared Memory (USM)**
Zugriff	über Accessor-Objekte mit Zugriffsmodus	über gewöhnliche Zeiger
Datentransfer	automatisch durch die Laufzeit	*shared*: automatisch; *device*: explizit mit `memcpy`
Abhängigkeiten	automatisch aus den Accessoren	selbst über Events oder in-order-Queue
Stärke	sicher, wenig Fehlerquellen	vertraut aus C++, leicht zu portieren

Tabelle 3: Die beiden Speichermodelle von SYCL

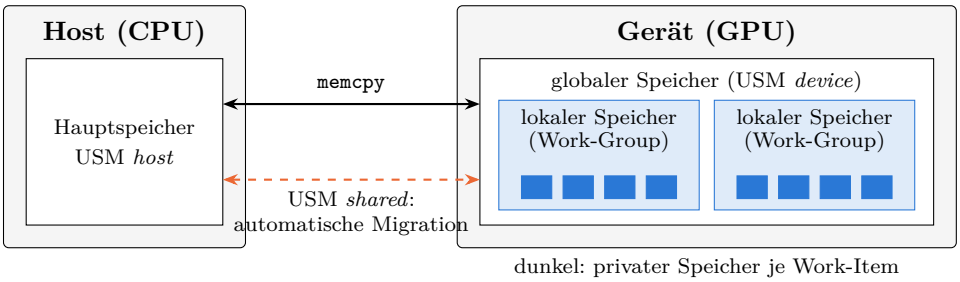


Abbildung 7: Speicherhierarchie von Host und Gerät und die Wege des Datentransfers

sowohl die nötigen Kopien als auch die Abhängigkeiten im Graphen ab. Die Daten gehören während der Lebensdauer des Buffers der Laufzeit; erst wenn der Buffer zerstört wird oder der Host über einen *Host-Accessor* zugreift, sind die Ergebnisse wieder im Hauptspeicher sichtbar (Abbildung 6). Dieses Modell ist sicher, verlangt aber ein Umdenken gegenüber gewöhnlichen Zeigern. Beim zeigerbasierten *USM*, das mit SYCL 2020 eingeführt wurde, wird Speicher ähnlich wie mit `malloc` angelegt. Es gibt drei Arten: *device* (nur auf dem Gerät), *host* (im Hauptspeicher, vom Gerät aus lesbar) und *shared* (auf beiden Seiten gültig, die Laufzeit migriert die Daten bei Bedarf) [12]. USM erleichtert die Portierung bestehender Programme, überträgt aber die Verantwortung für Abhängigkeiten auf den Programmierer.

Auf dem Gerät unterscheidet SYCL zudem (Abbildung 7) *globalen* Speicher (für alle Work-Items sichtbar, groß, aber langsam), *lokalen* Speicher (pro Work-Group, klein und schnell) und *privaten* Speicher (Register). Diese Hierarchie gezielt zu nutzen ist, wie bei CUDA, der wichtigste Hebel für Leistung [14].

5.5 Fehlerbehandlung, Spezialisierung und Interoperabilität

Fehler meldet SYCL als C++-Ausnahmen. Synchrone Fehler, etwa eine ungültige Work-Group-Größe beim Einreichen, werden sofort geworfen. Asynchrone Fehler, die erst wäh-

rend der Ausführung auf dem Gerät auftreten, sammelt die Laufzeit und übergibt sie einem *Async Handler*, sobald der Host wartet. Diese Trennung ist eine Folge der asynchronen Ausführung: Ein Fehler in einem Kernel kann erst sichtbar werden, wenn der Host längst weitere Aufträge eingereicht hat (Abbildung 8). *Spezialisierungskonstanten* erlauben es, Werte wie eine Kachelgröße erst zur Laufzeit festzulegen und dennoch vom Compiler als Konstante behandeln zu lassen, was zur JIT-Kompilierung passt. Über die *Backend-Interoperabilität* lassen sich schließlich native Objekte des Backends abrufen, um etwa herstellerspezifische Bibliotheken direkt auf SYCL-Daten aufzurufen [1].

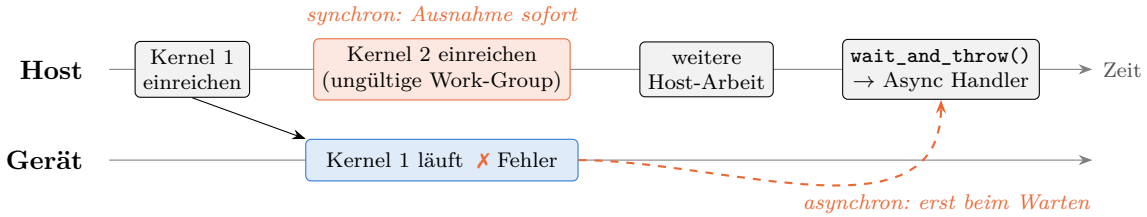


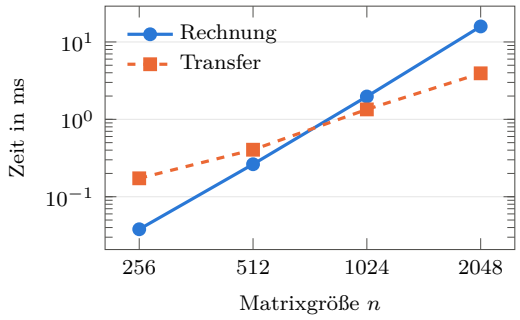
Abbildung 8: Synchrone Fehler werden beim Einreichen geworfen, asynchrone erst, wenn der Host wartet

6 Anwendungsbeispiel: Matrixmultiplikation

Zur Veranschaulichung wurde ein sequentielles C++-Programm für die Matrixmultiplikation $C = A \cdot B$ [14] schrittweise nach SYCL portiert und mit DPC++ auf einer CPU (AMD Ryzen 5 5600X) und einer GPU (NVIDIA GeForce RTX 5060 Ti) ausgeführt. Der Weg entspricht dem Anwendungsfall der Migration: Zuerst wurden die äußeren Schleifen durch ein `parallel_for` über USM *shared* ersetzt, wobei die Rechenvorschrift unverändert zum Kernel wurde. Danach wurden die Daten mit USM *device* explizit kopiert, und schließlich lud jede Work-Group Blöcke der Matrizen in den lokalen Speicher.

Alle Stufen liefen ohne Codeänderung auf beiden Geräten; gewechselt wurde nur das Gerät beim Programmstart. Abbildung 9 zeigt zwei Beobachtungen. Erstens kostet bei kleinen Matrizen der Datentransfer mehr als die Rechnung, weshalb das Überlappen über den Abhängigkeitsgraphen wichtig ist (a). Zweitens ist die beste Work-Group-Größe geräteabhängig: Auf der GPU ist eine Kachelgröße von $T = 16$ am schnellsten, auf der CPU $T = 32$, und bei $T = 64$ überschreitet die Work-Group die Grenze von 1024 Threads der NVIDIA-GPU, sodass der Kernel dort nicht startet (b). Die Kennzahl für Performance-Portabilität nach Pennycook et al. [15] fällt dann auf null.

(a) Rechnung und Transfer, GPU



(b) Effizienz je Kachelgröße, $n = 2048$

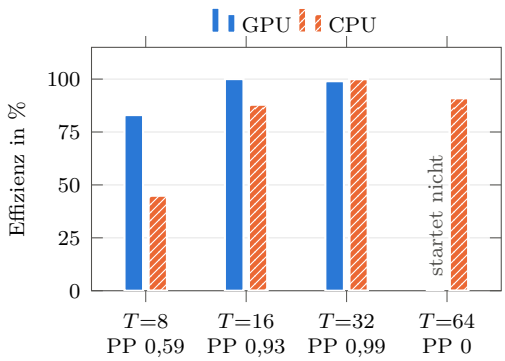


Abbildung 9: Datentransfer (a) und Leistung je Kachelgröße mit Performance-Portabilität (b)

7 Bewertung und Fazit

SYCL löst das Problem der Herstellerbindung und der mehrfachen Codebasen überzeugend: Ein Quellcode in Standard-C++ läuft auf CPUs und GPUs verschiedener Hersteller, und mehrere unabhängige Implementierungen sichern die Offenheit. Die Anwendungsfälle tragen dabei verschiedene Teile des Programmiermodells. Für die Migration aus CUDA sind die enge Verwandtschaft der Ausführungshierarchie und das zeigerbasierte USM entscheidend, für die gemeinsame Nutzung von CPU und GPU die asynchronen Queues mit ihrem Abhängigkeitsgraphen und für portable Bibliotheken Single-Source mit Templates sowie die Geräteabfragen.

Die Grenze liegt im dritten Problem, dem Zielkonflikt zwischen Abstraktion und Kontrolle. SYCL abstrahiert die Schnittstelle, nicht die Hardware: Für hohe Leistung muss der Programmierer Work-Groups und lokalen Speicher ebenso verstehen wie bei CUDA, und Parameter wie die Work-Group-Größe bleiben gerätespezifisch, wie das Beispiel zeigt. Hinzu kommen eine merkliche Abhängigkeit von der Implementierung [10] und eine anspruchsvolle Werkzeugkette.

Insgesamt erfüllt SYCL sein Versprechen „einmal schreiben, überall ausführen“ für die Korrektheit. Für hohe Leistung auf allen Geräten stellt es mit Geräteabfragen und Spezialisierungskonstanten die Werkzeuge bereit, nimmt dem Programmierer die gerätespezifische Abstimmung aber nicht ab.

Literaturverzeichnis

[1] The Khronos SYCL Working Group. „SYCL 2020 Specification," besucht am 30. Okt. 2026. Adresse: https://registry.khronos.org/SYCL/specs/sycl-2020/html/sycl-2020.html.

[2] J. Nickolls, I. Buck, M. Garland und K. Skadron, „Scalable Parallel Programming with CUDA," *ACM Queue*, Jg. 6, Nr. 2, S. 40–53, 2008. DOI: 10.1145/1365490.1365500.

[3] J. E. Stone, D. Gohara und G. Shi, „OpenCL: A Parallel Programming Standard for Heterogeneous Computing Systems," *Computing in Science & Engineering*, Jg. 12, Nr. 3, S. 66–73, 2010. DOI: 10.1109/MCSE.2010.69.

[4] Intel Corporation. „Intel LLVM-based Compiler (DPC++) – Repository," besucht am 30. Okt. 2026. Adresse: https://github.com/intel/llvm.

[5] A. Alpay und V. Heuveline, „SYCL beyond OpenCL: The Architecture, Current State and Future Direction of hipSYCL," in *Proceedings of the International Workshop on OpenCL (IWOCL '20)*, 2020. DOI: 10.1145/3388333.3388658.

[6] AdaptiveCpp Contributors. „AdaptiveCpp – Repository," besucht am 30. Okt. 2026. Adresse: https://github.com/AdaptiveCpp/AdaptiveCpp.

[7] A. Alpay und V. Heuveline, „One Pass to Bind Them: The First Single-Pass SYCL Compiler with Unified Code Representation Across Backends," in *Proceedings of the International Workshop on OpenCL (IWOCL '23)*, 2023.

[8] H. C. Edwards, C. R. Trott und D. Sunderland, „Kokkos: Enabling Manycore Performance Portability through Polymorphic Memory Access Patterns," *Journal of Parallel and Distributed Computing*, Jg. 74, Nr. 12, S. 3202–3216, 2014. DOI: 10.1016/j.jpdc.2014.07.003.

[9] D. A. Beckingsale u. a., „RAJA: Portable Performance for Large-Scale Scientific Applications," in *2019 IEEE/ACM International Workshop on Performance, Portability and Productivity in HPC (P3HPC)*, 2019, S. 71–81.

[10] T. Deakin und S. McIntosh-Smith, „Evaluating the Performance of HPC-Style SYCL Applications," in *Proceedings of the International Workshop on OpenCL (IWOCL '20)*, 2020. DOI: 10.1145/3388333.3388643.

[11] M. Breyer, A. Van Craen und D. Pflüger, „A Comparison of SYCL, OpenCL, CUDA, and OpenMP for Massively Parallel Support Vector Machine Classification on Multi-Vendor Hardware," in *Proceedings of the International Workshop on OpenCL (IWOCL '22)*, 2022.

[12] J. Reinders, B. Ashbaugh, J. Brodman, M. Kinsner, J. Pennycook und X. Tian, *Data Parallel C++: Programming Accelerated Systems Using C++ and SYCL*, 2. Aufl. Berkeley, CA: Apress, 2023, ISBN: 978-1-4842-9691-2. DOI: 10.1007/978-1-4842-9691-2.

[13] NVIDIA Corporation. „CUDA C++ Programming Guide," besucht am 30. Okt. 2026. Adresse: https://docs.nvidia.com/cuda/cuda-c-programming-guide/.

[14] W.-m. W. Hwu, D. B. Kirk und I. El Hajj, *Programming Massively Parallel Processors: A Hands-on Approach*, 4. Aufl. Cambridge, MA: Morgan Kaufmann, 2022.

[15] S. J. Pennycook, J. D. Sewall und V. W. Lee, „A Metric for Performance Portability,“ in *Proceedings of the 7th International Workshop on Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems (PMBS)*, 2016. arXiv: 1611.07409.